

Assignment PART A

1. Difference between `df.loc[]` and `df.iloc[]`

- `df.loc[]` uses labels/names to select data while `df.iloc[]` uses integer positions/index numbers
- `df.loc[]` Label-based indexing while `df.iloc[]` is position-based indexing
- `df.loc[]` can use row/column names while `df.iloc[]` uses numerical positions
- Example: `df.loc[2, "age"]`. While Example: `df.iloc[2, 1]`

2. Trade-off between `fillna()` and `dropna()`

When dealing with missing data, you generally have two choices:

- `df.fillna()`
`fillna()` replaces missing values with another value, such as the mean, median, or a specific value.

Example: `df["age"] = df["age"].fillna(df["age"].mean())`

Advantages:

Keeps all the observations/rows.

Prevents losing potentially useful data.

Useful when the dataset is small.

Disadvantage:

The value used for replacement is an estimate and may not represent the actual missing value.

Poor imputation choices can introduce bias or distort the data.

- `df.dropna()`
`dropna()` removes rows or columns containing missing values.

Example:

`df = df.dropna()`

Advantages:

Simple and straightforward.

Does not introduce estimated values.

Disadvantages:

You lose observations.

If many values are missing, the dataset can become much smaller.

Removing data can potentially introduce bias.

Main trade-off:

`fillna()` preserves data but introduces estimated values, while `dropna()` removes incomplete data but can result in loss of useful information.